第六章作业

1 有序树的表示、树高与"长子—右兄弟"转换

已知一棵有序树 `T` 的父子关系如下（同一行中的孩子按从左到右排列）：

```
R -> A, B, C
A -> D, E
B -> F
C -> I, J
E -> G, H
J -> K
```

请完成下列问题：

1. 写出该树的树形（缩进）表示和括号表示法。

   树形（缩进）表示：

   ```
   R
     A
       D
       E
         G
         H
     B
       F
     C
       I
       J
         K
   ```

   括号表示法：

   ```
   R(A(D,E(G,H)),B(F),C(I,J(K)))
   ```

2. 求该树的高度（规定根结点位于第 1 层），并写出递归求树高的伪代码。

该树的高度为 4。

```
"""
# Definition for a Node.
class Node:
    def __init__(self, val: Optional[int] = None, children: Optional[List['Node']]
```

```
= None):
        self.val = val
        self.children = children
"""

class Solution:
    def maxDepth(self, root: 'Node') -> int:
        if not root: return 0
        if not root.children: return 1
        return max([self.maxDepth(child) for child in root.children]) + 1
```

3. 使用"长子—右兄弟"法将该树转换为二叉树，写出转换后二叉树中每个非空的"左孩子"和"右孩子"关系，并给出该二叉树的先序遍历序列。

节点	左孩子	右孩子
R	A	null
A	D	B
D	null	E
E	G	null
G	null	H
H	null	null
B	F	C
F	null	null
C	I	null
I	null	J
J	K	null
K	null	null

先序遍历序列： R, A, D, E, G, H, B, F, C, I, J, K

> 第 (2) 问求树高对应编程题：力扣 559. N 叉树的最大深度。第 (3) 问对应会员题力扣 431. 将 N 叉树编码为二叉树。
>
> **E559.N 叉树的最大深度**
>
> https://leetcode.cn/problems/maximum-depth-of-n-ary-tree/

---

2 完全 K 叉树的结点数与编号关系

设有一棵高度为 `H` 的完全 `K` 叉树，其中根结点位于第 0 层，最深结点位于第 `H` 层。各结点按层序从 0 开始连续编号，树中共有 `N` 个结点，且 `K >= 2`。

请回答：

1. `N` 的取值范围。

可知 $\sum_{i=0}^{H-1} K^i < N \le \sum_{i=0}^{H} K^i$，即是 $\frac{K^H-1}{K-1} < N \le \frac{K^{H+1}-1}{K-1}$。

2. 编号为 `n` 的非根结点的父结点编号。

对于编号为 `n` 的非根结点，其父结点的编号为 $\left\lfloor \frac{n-1}{K} \right\rfloor$。

3. 编号为 `n` 的结点的第 `i` 个子结点编号，其中 `i = 1, 2, ..., K`；同时说明该编号何时确实对应树中的结点。

编号为 `n` 的结点的第 `i` 个子结点编号为 $n \times K + i$，其中 $i = 1, 2, ..., K$。该编号确实对应树中的结点当且仅当 $n \times K + i < N$。

4. 编号为 `n` 的非根结点在什么条件下有右兄弟？若存在，右兄弟的编号是多少？

编号为 `n` 的非根结点有右兄弟当且仅当 $n + 1 < N$ 且 $\left\lfloor \frac{n-1}{K} \right\rfloor = \left\lfloor \frac{n}{K} \right\rfloor$。若存在，右兄弟的编号是 $n + 1$。

---

3 按规模合并 + 路径压缩的并查集

初始有 12 个元素 `0` 到 `11`，每个元素各自构成一棵独立的树。并查集采用以下规则：

- `union(x, y)` 先对 `x`、`y` 执行带路径压缩的 `find`，再将规模较小的树根挂到规模较大的树根之下。
- 两棵树规模相同时，将 `y` 所在树的根挂到 `x` 所在树的根之下。
- 根结点的父结点是它自身。

依次执行：

```
union(0, 1)
union(2, 3)
union(4, 5)
union(6, 7)
union(0, 2)
union(4, 6)
union(8, 9)
union(10, 11)
union(8, 10)
find(3)
union(6, 8)
union(3, 5)
find(1)
find(11)
```

请完成下列问题：

1. 写出所有操作完成后的 `parent[0..11]` 数组。注意：只有实际执行过 `find` 的路径会被压缩，不要把所有结点自动改为直接指向最终根。

0	1	2	3	4	5	6	7	8	9	10	11
4	4	0	0	4	4	4	6	4	8	4	4

2. 说明采用"按规模合并"时，为什么树的高度为 $O(\log N)$。

可以发现对于结点 x，只有在其所在的树规模较小，被挂在一个更大规模的树下的时候，x 的深度才会增加 1，因此每次 x 的深度增加 1 时，其所在的树规模至少翻倍。假设 x 的深度为 d，则其所在的树规模至少为 2^d，而树的规模不超过 N，因此有 $2^d \leq N$，即 $d \leq \log_2 N$。

> 并查集练习题：力扣 547. 省份数量。
>
> **M547.省份数量**
>
> https://leetcode.cn/problems/number-of-provinces/

```python
class Solution:
    def findCircleNum(self, isConnected: List[List[int]]) -> int:
        n = len(isConnected)
        parent = list(range(n))
        def find(i: int) -> int:
            if parent[i] != i:
                parent[i] = find(parent[i])
            return parent[i]
        def union(i1: int, i2: int):
            parent[find(i1)] = find(i2)
        for i in range(n):
            for j in range(i + 1, n):
                if isConnected[i][j] == 1:
                    union(i, j)
        return sum(parent[i] == i for i in range(n))
```